

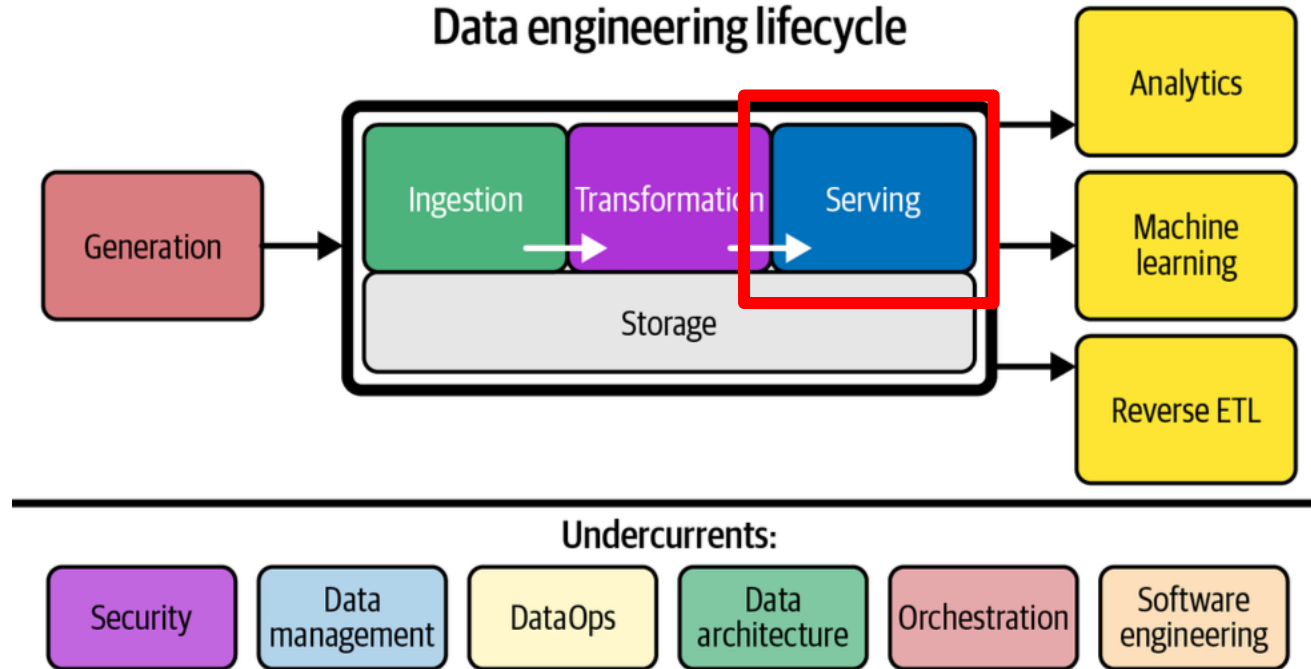
Data Engineering

Serving

UA.DETI.ED - 2025/26

José Luís Oliveira

The Data Engineering Lifecycle



Contents

- ❖ The Foundations
 - The importance of trust, data observability, and identifying the right use cases and users to build effective data products.
- ❖ The Evolution of Serving
 - Moving from passive business analytics to real-time operational and embedded analytics.
- ❖ Modern Architectures
 - The rise of Universal Semantic Layers (Headless BI) and decentralized serving via the Data Mesh paradigm.
- ❖ The AI Era
 - Utilizing Vector Databases for similarity searches and integrating Generative AI (RAG pipelines, autonomous code generation) with numerical data.
- ❖ Serving Mechanisms
 - Using cloud OLAP databases, query federation, secure cloud data sharing, and serving data directly to data scientists in notebooks.
- ❖ Operationalizing Data
 - Pushing analytical data back into source systems via Reverse ETL and Composable CDPs.
- ❖ Undercurrents
 - Enforcing strict security/privacy (least privilege), managing DataOps (monitoring data health), and optimizing FinOps for cloud queries.

Introduction to the Serving Stage

- ❖ Serving is the final stage where data engineering delivers tangible value to consumers and stakeholders.
 - It prepares and delivers data for critical applications including business intelligence (BI) analytics, machine learning training, and reverse ETL workflows.
- ❖ This stage focuses on providing data in the specific formats, structures, and latencies expected by downstream users.
 - It represents the final opportunity to ensure data quality and build trust with the organization.
- ❖ A successful serving layer transforms technical outputs into business-ready information.

The Importance of Trust and Data Observability

- ❖ Trust is the root consideration when serving data; without it, even the most sophisticated data products will go unused by stakeholders.
 - A loss of trust acts as a silent death knell for a data project, as credibility is insanely difficult to earn back once it has been lost.
- ❖ Data engineers must proactively build and maintain trust through rigorous data validation and data observability processes, in conjunction with visually inspecting and confirming validity with stakeholders
 - Data validation is analysing data to ensure that it accurately represents financial information, customer interactions, and sales.
 - Data observability provides an ongoing view of data and data processes.
- ❖ Establishing clear Service-Level Agreements (SLAs) and Service-Level Objectives (SLOs) ensures that users have consistent expectations regarding data availability and quality.

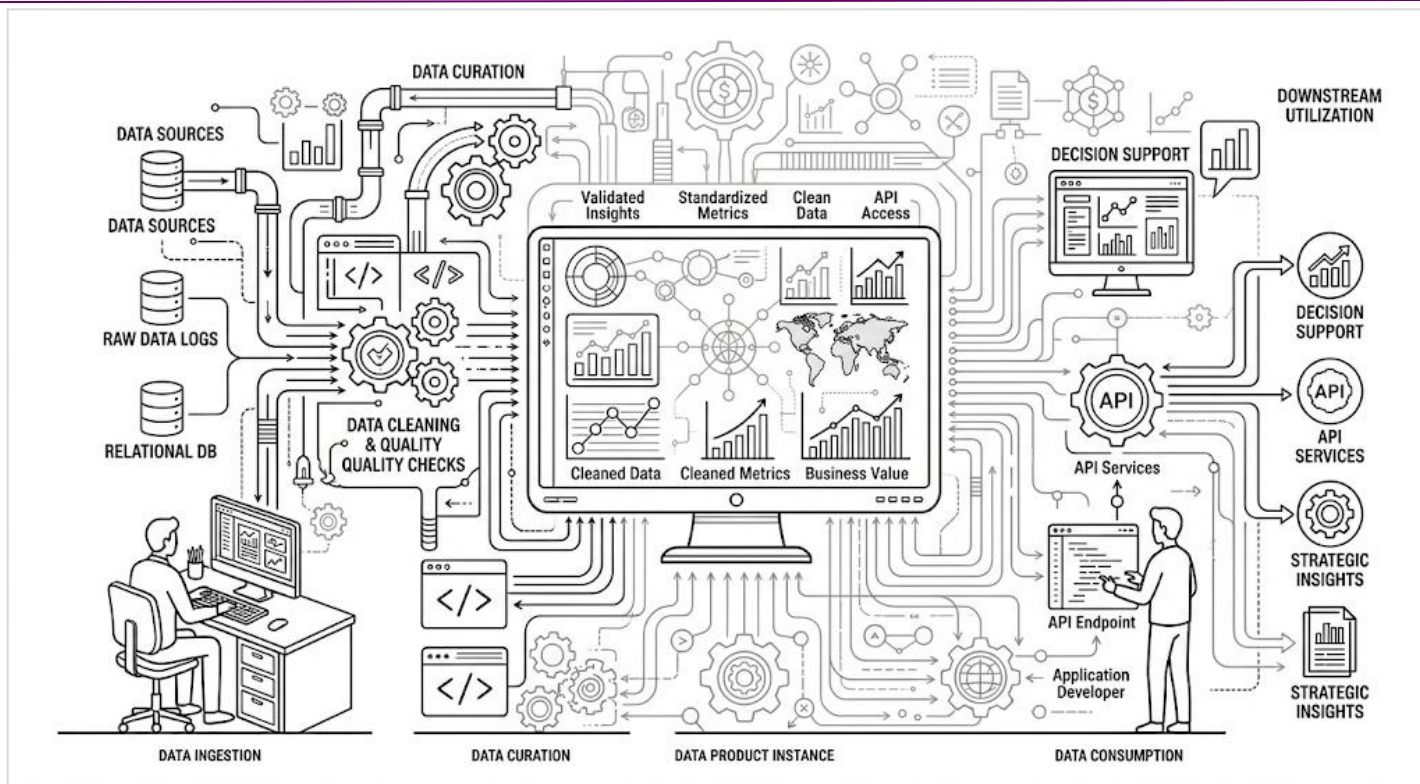
Identifying the Use Case and User

- ❖ Data is most valuable when it leads to direct, automated actions rather than simply populating passive reports.
 - Before building a pipeline, data engineers should always ask what action the data will trigger and who will be performing that action.
- ❖ Working backward from the user's expectations ensures a much better product-market fit.
 - Engineers should collaborate closely with stakeholders to prioritize use cases that offer the highest possible Return on Investment (ROI) for the business.
- ❖ When starting a new data project, while it's tempting to focus on tools, you must start with the use case and the users, e.g.:
 - Who will use the data, and how will they use it?
 - What do stakeholders expect?
 - How can I collaborate with data stakeholders (e.g., data scientists, analysts, business users) to understand how the data I'm working with will be used?

Building Effective Data Products

- ❖ A data product is defined as a tool or asset that facilitates an end goal through the use of data.
 - Building them successfully requires deeply involving key stakeholders to understand the specific "job to be done" by the end user.
- ❖ Highly successful data products naturally create positive feedback loops.
 - Increased usage generates more useful data, which in turn is continuously used to iteratively improve the data product itself.

Building Effective Data Products



The Traditional Era: Passive Business Analytics (Serving Humans)

- ❖ Historically, the primary use case for serving data was business analytics, which relies on historical data to drive strategic, long-term decisions.
- ❖ Workflow:
 - Data engineers built highly structured data warehouses and data marts. Analysts would then query this data to build dashboards and reports.
- ❖ Limitation:
 - This approach was inherently passive. A dashboard can show an executive that sales are down, but a human still has to interpret that chart and manually take action. Furthermore, definitions of metrics were often buried in fragmented, inconsistent SQL scripts across different departments.

Serving for Business Analytics

- ❖ Business analytics utilizes historical and current data to drive strategic, long-term actionable decisions.
 - This serving area includes developing executive dashboards for core metrics, generating targeted reports, and conducting exploratory ad hoc analysis.
- ❖ Good analysts dive deeply into data to uncover hidden trends and provide crucial feedback on data quality.
 - Data engineers support these efforts by acquiring requested datasets and serving them reliably through modern data warehouses or data lakes.
- ❖ Examples:
 - *A weekly executive dashboard connecting directly to a data warehouse to show total sales revenue per region compared to the previous quarter.*
 - *A data analyst querying historical transaction data to generate an ad-hoc report investigating why a specific product's return rate spiked last month.*

Operational Analytics & Real-Time Action

- ❖ Operational analytics leverages real-time data streams to trigger immediate, impactful actions.
 - Examples include real-time application monitoring that triggers automated alerts or auto-scaling events when specific performance thresholds are breached.
- ❖ The line between business and operational analytics continues to blur.
 - Modern architectures now combine historical trends with up-to-the-second streaming events, demanding platforms that can handle both cold and red-hot data in one place.

Embedded & User-Facing Analytics

- ❖ Embedded analytics provides real-time data insights directly to external customers within software applications.
 - For instance, a smart thermostat app showing real-time power consumption, or a SaaS platform displaying live inventory metrics to its third-party sellers.
- ❖ Serving user-facing analytics requires architectures built for extreme performance.
 - These systems must handle high concurrency, custom multi-dimensional queries, and ultra-low data latency to ensure millisecond response times.

The Standardization Era: Universal Semantic Layers (Headless BI)

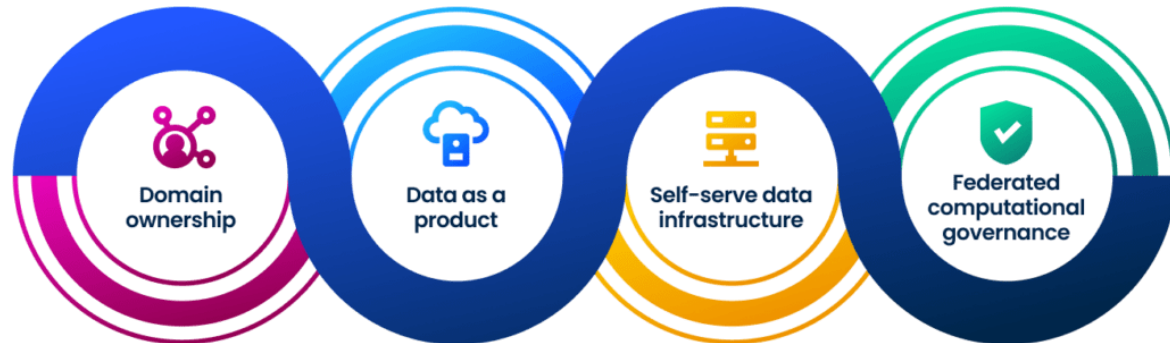
- ❖ As data ecosystems grew more complex, a major problem emerged: different tools and teams were calculating metrics differently.
 - E.g., If the marketing dashboard and the finance dashboard report different numbers for "Revenue," trust in the data evaporates.
- ❖ The Solution:
 - The introduction of the **Universal Semantic Layer** (or metrics layer).
- ❖ How it Works:
 - Instead of encoding business logic inside individual BI tools, organizations now centrally maintain and compute business logic independent of the storage or presentation layer.
 - Tools like *dbt* or *LookML* allow data teams to define a metric once. Whether a user queries the data, the semantic layer intercepts the request and ensures everyone gets the exact same answer.

The Decentralization Era: Data Mesh

- ❖ Instead of a bottlenecked, centralized data engineering team serving the entire company, organizations are moving toward decentralized architectures like the Data Mesh.
- ❖ Domain Ownership:
 - In a data mesh, domain teams (e.g., the Marketing tech team or the Supply Chain tech team) are directly responsible for preparing and serving their own high-quality "data products" to the rest of the organization.
- ❖ Peer-to-Peer Serving:
 - This shifts data serving into a scalable, peer-to-peer model where teams can seamlessly consume specialized datasets generated by other domain experts.

Decentralized Serving with Data Mesh

- ❖ The data mesh paradigm fundamentally changes how data is served by moving away from highly siloed, centralized data teams.
 - Domain teams become directly responsible for preparing and serving high-quality data products to other teams across the organization.
- ❖ This approach promotes a scalable, peer-to-peer serving model.
 - Teams can run their own self-service analytics while simultaneously consuming specialized data products generated by other domain experts.





❖ 1. Domain-Oriented Data Ownership

- This principle decentralizes data ownership from a single, centralized data team to the specific business domains (e.g., Marketing, Sales, Finance, Engineering) that generate or consume the data.
- Instead of sending all data to a monolithic central data lake or warehouse, each domain team is responsible for managing its own storage, pipelines, and data quality.

❖ 2. Data as a Product

- Under this philosophy, domain teams treat their analytical data as a product that they provide to the rest of the organization, much like a public API.
- To be successful, these data products must be highly refined, easily discoverable (via a data catalog), addressable, interoperable, secure, and trustworthy.



❖ 3. Self-Serve Data Infrastructure Platform

- Because domain teams (like Sales or Marketing) may not have deep data engineering expertise, a dedicated central data platform team provides them with domain-agnostic infrastructure, tools, and systems.
- This self-serve "machine" empowers domain teams to easily build, store, execute, and maintain their own data products without bottlenecking on a central IT team.

❖ 4. Federated Computational Governance

- To prevent the decentralized domains from turning into disconnected data silos, a federated governance model is established to ensure universal interoperability and standardization.
- A governance group creates global policies (such as security rules, PII anonymization, and data quality standards), which are then automatically enforced and executed across all domains by the underlying self-serve data platform.

The AI Era: Vector Databases & GenAI (Serving Machines)

- ❖ The most recent evolution in data serving is designed for Generative AI and Large Language Models (LLMs).
- ❖ LLMs need enterprise context to provide accurate answers, and the serving layer had to adapt to provide it.
- ❖ Vector Databases:
 - Traditional databases serve rows and columns. Vector databases serve high-dimensional mathematical representations (embeddings) of unstructured data like text documents, PDFs, and images.
- ❖ Retrieval-Augmented Generation (RAG):
 - The serving layer is now responsible for RAG pipelines. When a user asks an AI chatbot a question, the serving layer instantly searches the vector database for relevant enterprise documents, and "serves" that context to the LLM so it can generate an accurate, hallucination-free answer.
- ❖ AI Meets the Semantic Layer:
 - AI agents are now being plugged directly into Universal Semantic Layers. Because the semantic layer translates natural language into strictly governed SQL, business users can safely chat with their enterprise data without the AI hallucinating incorrect database queries.

The Rationale for Vector Databases (Similarity vs. Exact Match)

- ❖ Traditional databases are built for exact match queries
 - `SELECT * FROM transactions WHERE amount > 500`).
- ❖ Vector databases are built for similarity searches in high-dimensional space.
 - If you take a user's numerical data you can represent that user as a single mathematical vector (an array of numbers).
- ❖ Recommendation Engines:
 - Instead of writing complex SQL joins to find users who bought similar things, you ask the vector database to find the vectors that are mathematically "closest" to your target user.
- ❖ Anomaly and Fraud Detection:
 - If a credit card normally makes transactions that cluster in a specific area of your vector space, and suddenly a transaction appears that is mathematically "far away" from that cluster, the vector database instantly flags it as an anomaly. This is much faster and more nuanced than writing static IF/THEN rules.

The Rationale for GenAI with Numbers

(The Reasoning Engine)

- ❖ Large Language Models (LLMs) can be used to *reason about* and *translate* numerical data. Examples:
 - ❖ Data-to-Text Translation
 - Imagine an IoT sensor on a factory loom that is streaming numerical data about temperature, pressure, and speed. If a vector database flags an anomaly in those numbers, an LLM can be triggered to look at the numerical shift and instantly generate a plain-English diagnostic report for the floor manager (e.g., "*Sensor 4 indicates a 15% drop in pressure over the last hour, likely indicating a valve leak.*").
 - ❖ Autonomous Code Generation
 - Instead of having an analyst write Python scripts to visualize a numerical dataset, modern GenAI agents can look at the metadata and schema of your numerical database, write the required Python/Pandas code in the background, execute it, and serve the resulting graph directly to the user.

Semantic Layers and GenAI Integration

- ❖ Semantic layers have become the critical bridge enabling Large Language Models (LLMs) to accurately query enterprise data.
 - By exposing strict, well-documented metric definitions to AI agents, organizations prevent LLMs from hallucinating incorrect SQL queries or business facts.
- ❖ This architecture allows business stakeholders to use natural language conversational interfaces to securely explore data.
 - The semantic layer acts as the guardrail, translating the AI's natural language request into optimized, compliant database queries.

Serving Data for Machine Learning

- ❖ Machine learning applications rely entirely on the consistent delivery of appropriately prepared, high-quality data.
 - Data engineers work closely with data scientists to build complex featurization pipelines and automate data delivery for model training.
- ❖ Understanding core ML concepts helps data engineers design much more scalable serving systems.
 - Familiarity with the differences between batch and online learning, and the nuances of structured versus unstructured data, is essential.
- ❖ Example:
 - *An automated data pipeline that continuously feeds cleaned user clickstream data and demographic features into a model training environment to update an e-commerce product recommendation engine.*

Serving Data from Cloud Databases

- ❖ OLAP databases, including data warehouses and data lakes, are critical foundational layers for serving analytics and ML workflows.
 - They impose necessary order through schemas and offer fine-grained access controls at the table, column, and row levels.
- ❖ Modern cloud databases strictly separate compute from storage, drastically altering cost and performance dynamics.
 - Data teams can dynamically spin up isolated, dedicated data warehouse clusters for different workloads, preventing resource contention and allowing for precise budgeting.

Query Federation in the Modern Data Stack

- ❖ Query federation allows an OLAP engine to query and join data from multiple disparate sources without needing to physically centralize the data first.
 - Distributed query engines can securely serve queries that span across cloud object storage and remote RDBMS environments simultaneously.
- ❖ Federation provides excellent flexibility for rapid exploratory analysis.
 - It also greatly enhances security by providing read-only access to source systems without continuously duplicating sensitive data into new repositories.

Secure Data Sharing in the Cloud

- ❖ Modern cloud platforms enable massively multitenant data sharing without the need to copy or move physical files over insecure channels.
 - Organizations can securely share specific, filtered datasets with internal domain teams or external partner businesses directly through the platform.
- ❖ Data sharing effectively shifts data serving into an access control paradigm.
 - The actual query processing overhead is handled and paid for by the data consumers, dramatically streamlining the data engineer's operational burden.

Serving Data in Notebooks & Data Science

- ❖ Data scientists heavily rely on notebooks like Jupyter to explore data, engineer features, and train models locally or in the cloud.
 - Data engineers assist by configuring programmatic connections to APIs, data warehouses, and object storage systems within these environments.
- ❖ Scaling notebook environments often requires moving workloads from local machines to robust cloud infrastructure.
 - Cloud-based notebooks and distributed execution systems, such as Spark or Dask, handle datasets that vastly exceed a local machine's memory constraints.
- ❖ Example:
 - *A data scientist using a cloud-based Jupyter Notebook to securely query an API and a data lake, loading a massive 50GB dataset into a distributed Spark cluster to explore the data without crashing their local laptop.*

Secure Credential Handling

- ❖ Incorrectly handled credentials in data science notebooks pose a massive security vulnerability for the organization.
 - It is dangerously common for data scientists to embed passwords directly into code, which subsequently leaks into version control repositories.
- ❖ Data engineers must establish and strictly enforce standards for credential management.
 - Data scientists should be provided with, and trained to use, secure credential managers, environment variables, or SSO-based command-line tools.

The Operational Era: Reverse ETL & Active Data (Serving Applications)

- ❖ As companies demanded faster ROI from their data, the serving layer evolved from passive insights to active operations. This era blurred the line between business analytics and operational analytics (taking immediate action based on data).
- ❖ Reverse ETL:
 - Instead of trapping valuable insights inside a data warehouse dashboard, Reverse ETL pushes processed, analytical data directly back into operational source systems.
 - For example, a machine learning model might score customer leads in the warehouse, and Reverse ETL syncs those scores directly into the sales team storage, to act on immediately.
- ❖ The Composable CDP:
 - This trend turned the cloud data warehouse into the engine of the "Composable Customer Data Platform," allowing marketing and support teams to trigger automated workflows based on real-time data.

Reverse ETL: Operationalizing Data

- ❖ Reverse ETL is the process of extracting processed, analytical data from a data warehouse and loading it back into operational source systems.
 - An example includes pushing a machine learning lead-scoring model directly back into a CRM for the sales team to utilize immediately.
- ❖ It significantly reduces friction by delivering actionable data directly to the platforms where end-users already work.
 - Engineers must be extremely careful to build monitoring and guardrails, as reverse ETL inherently creates risky feedback loops in automated systems.

Reverse ETL & Composable CDPs

- ❖ Reverse ETL tools have matured into the backbone of the "Composable Customer Data Platform (CDP)."
 - Instead of buying a rigid, off-the-shelf CDP, organizations now use the data warehouse as the single source of truth and Reverse ETL to activate customer segments.
- ❖ This architecture empowers marketing and support teams with real-time operational analytics.
 - Automated pipelines now feature automatic Personally Identifiable Information (PII) detection and consent tracking to ensure global data privacy compliance before syncing.

Security and Privacy in the Serving Layer

- ❖ The serving stage presents the absolute largest security surface in the entire data engineering lifecycle.
 - Sharing data indiscriminately without fine-grained access controls can lead to catastrophic data breaches, regulatory fines, and bad press.
- ❖ The principle of least privilege must be rigorously exercised for both human users and automated service accounts.
 - Access should be narrowed at the row, column, and cell levels, and data traversing the public internet must always be heavily encrypted.

DataOps & FinOps in Data Serving

- ❖ DataOps operationalizes data management by actively monitoring data health, downtime, and pipeline latency at the serving layer.
 - Observability tools help data engineers ensure that dashboards and ML models are not being fed stale, anomalous, or incomplete data.
- ❖ As cloud data warehouses operate on consumption-based pricing, FinOps has become crucial.
 - Data engineers are now responsible for actively monitoring and optimizing query costs, ensuring that computationally expensive ML models or BI dashboards do not cause budget overruns.

Conclusion: The Feedback Loop

- ❖ The data engineering lifecycle has a logical ending at the serving stage, but it fundamentally operates as a continuous feedback loop.
 - Data engineers should view the serving stage as a critical opportunity to learn what's working and identify upstream bottlenecks.
- ❖ Actively listen to downstream stakeholders, analysts, and business users.
 - Treat user feedback not as criticism, but as vital intelligence needed to continually iterate and improve the organization's data products.

Further Reading & Resources

- ❖ Reis & Housley, Fundamentals of Data Engineering,
 - Chapter 9
 - <https://learning.oreilly.com/library/view/fundamentals-of-data/9781098108298/>
- ❖ Also deepen your understanding of modern data serving, semantic layers, and GenAI data architectures

